DJANGO TEXTBOOK SERIES [FILE 02] · ESSENTIAL TERMINOLOGY & SYNTAX GLOSSARY (LECTURES 1–11)

[2번째 파일] 베테랑 개발자가 정리한 Django 필수 용어 암기집

교재 1권~10권에 등장하는 130대 필수 개발 용어 · 내장 함수 · DTL 태그/필터 · 필드 옵션 · ORM 메서드 · 인증 및 암호화 보안 개념을 10대 테마별로 완벽 정리한 사전

본 문서는 Django 3종 학습 시리즈 중 **2번째 파일(필수 용어 암기집)**입니다. 원리를 이해했더라도 정확한 용어 명칭, 명령어 철자, 옵션 이름, 내장 클래스와 함수 이름이 손끝에서 바로 나오지 않으면 코드를 작성하거나 시험 문제를 풀 때 막히게 됩니다. 전 11권의 필수 용어를 10개 테마로 분류하고, **[정확한 정의 + 실전 코드 문법 + 베테랑의 1줄 암기 공식]**을 짝지어 수록했습니다.

테마 01 : 웹 아키텍처 · 가상환경 · 프로젝트 설정 필수 용어 (교재 1권)

필수 용어 / 파일명	핵심 정의 및 역할 (What & Why)	정확한 명령어 / 코드 예시	베테랑 1줄 암기 공식
Client (클라이언트)	서버에 서비스를 요청(Request)하고 응답 결과를 화면에 그려주는 주체 (웹 브라우저, 모바일 앱)	Chrome, Safari, 모바일 앱	“주문을 넣는 손님”
Server (서버)	클라이언트의 요청을 받아 DB 작업 및 비즈니스 로직을 처리한 뒤 응답(Response)을 돌려주는 컴퓨터/프로그램	Django 웹 서버	“요리를 만들어 내오는 주방”
Frontend vs Backend	• **Frontend**: 사용자에게 보이는 UI 화면 영역 • **Backend**: 뒤편에서 데이터 처리·DB·보안을 담당하는 영역	Front: HTML, CSS, JS, Vue.js Back: Python, Django	“홀 서빙(Front) vs 주방(Back)”
Framework (프레임워크)	웹 개발에 필요한 기본 구조와 뼈대, 규칙을 미리 만들어 둔 소프트웨어 개발 환경 (라이브러리들의 거대한 집합체)	Django, FastAPI, Spring	“밀키트와 주방 설비가 갖춰진 식당”
LTS (Long Term Support)	일반 버전보다 훨씬 긴 기간(보통 3년 이상) 동안 보안 패치와 버그 수정을 보증하는 장기 지원 안정 버전	Django 4.2 LTS, 5.2 LTS	“실무 기업이 채택하는 안정 보증판”
Virtual Environment (`venv`)	한 컴퓨터 안에서 프로젝트별로 파이썬 패키지 버전이 충돌하지 않도록 격리된 독립 실행 공간을 만드는 기술	`python -m venv venv` `source venv/Scripts/activate`	“집 안에 방음되는 독립된 방 짓기”
`requirements.txt`	현재 가상환경에 설치된 모든 패키지 이름과 버전을 텍스트로 기록한 의존성 명세서 파일	추출: `pip freeze > requirements.txt` 설치: `pip install -r requirements.txt`	“방 통째로 말고 가구 목록표만 공유!”
`.gitignore`	Git 버전 관리에서 추적 및 업로드하지 않을 파일·폴더 목록을 지정하는 설정 파일	`venv/`, `db.sqlite3`, `__pycache__/`	“무거운 가상환경과 로컬 DB는 제외!”
Project vs App	• **Project**: 전체 웹 서비스 단위 (설정·루트 URL 포함) • **App**: 게시글, 회원 등 독립된 단일 기능 모듈 단위	`django-admin startproject pjt .` `python manage.py startapp articles`	“백화점 건물(Project)과 입점 매장(App)”
MTV 디자인 패턴	Django의 역할 분리 패턴. **Model**(DB 관리) + **Template**(화면 UI, MVC의 View) + **View**(중간 제어 사령탑, MVC의 Controller)	`models.py` / `*.html` / `views.py`	“MVC의 View=Template, Controller=View!”
`settings.py`	프로젝트의 모든 환경설정(앱 등록, 템플릿 경로, DB, 언어, 시간대, 커스텀 유저 모델 등)을 총괄하는 파일	`INSTALLED_APPS`, `TEMPLATES`, `LANGUAGE_CODE = 'ko-kr'`	“백화점 중앙 통제실 설정판”
`manage.py`	프로젝트와 상호작용(서버 실행, 앱 생성, 마이그레이션, 관리자 생성 등)하는 명령줄 유틸리티 스크립트	`python manage.py runserver`	“절대 직접 수정하지 않는 만능 리모컨”

테마 02 : URL Dispatcher · View · HTTP 통신 필수 용어 (교재 1·2·6권)

필수 용어 / 함수명	핵심 정의 및 역할 (What & Why)	정확한 명령어 / 코드 예시	베테랑 1줄 암기 공식
URLconf (`urls.py`)	클라이언트가 요청한 URL 경로와 이를 처리할 View 함수를 1:1로 매핑해 주는 안내데스크 (URL 분배기)	`path('index/', views.index)`	"주소 보고 담당 부서로 연결하는 교환원"
`include()`	루트 `urls.py`에서 특정 접두사(예: `articles/`)까지 일치하면 앞부분을 잘라내고 나머지 경로를 앱의 `urls.py`로 위임하는 함수	`path('articles/', include('articles.urls'))`	"앞 주소는 가위로 자르고 앱에게 토스!"
Variable Routing (변수 라우팅)	URL 경로의 일부를 변수로 받아 View 함수의 동일한 이름의 매개변수로 자동 전달하는 기법	`path('<int:pk>/', views.detail)` → `def detail(request, pk):`	"Path Converter 기본값은 `str`, 숫자는 `int`!"
URL Namespace (`app_name` & `name`)	URL에 앱 성씨(`app_name`)와 별명(`name`)을 붙여 템플릿이나 View에서 주소를 하드코딩하지 않고 역참조하게 해주는 기능	`app_name = 'articles'` `{% url 'articles:detail' article.pk %}`	"주소가 바뀌어도 이름표로 부르면 안전!"
`request` 객체	클라이언트의 모든 요청 정보(전송 방식 `method`, 입력 데이터 `GET`/`POST`, 로그인 사용자 `user`, 쿠키 등)를 담은 첫 번째 필수 인자	`def index(request):`	"모든 View 함수의 1번 인자는 무조건 `request`!"
Query String (쿼리 스트링)	URL 주소 뒤에 물음표(`?`)를 붙이고 `key=value&...` 형태로 서버에 추가 파라미터를 전달하는 문자열	`/search/?q=django&page=2` → `request.GET.get('q')`	"HTML `<input name='q'>`가 Key가 됨!"
`render()` vs `redirect()`	• `render`: 템플릿 HTML에 `context` 데이터를 합성해 `200 OK` 응답 반환 • `redirect`: 지정한 URL로 다시 요청하라고 `302 Found` 이동 명령 반환	`render(request, 'a.html', context)` `redirect('articles:detail', pk)`	"조회 끝엔 `render`, 생성·수정·삭제 끝엔 `redirect`!"
PRG 패턴 (Post-Redirect-Get)	`POST` 요청 처리 직후 바로 `render()`하지 않고 `redirect()`로 새 `GET` 요청을 유도해 새로고침(F5) 중복 제출을 막는 설계 패턴	`POST /create/` → `302` → `GET /1/`	"마지막 요청 기록을 안전한 `GET`으로 세탁!"

테마 03 : DTL(Django Template Language) 필수 태그 & 필터 암기 사전 (교재 2권)

DTL 태그 / 필터 문법	핵심 역할 및 출력 결과	실전 코드 예시 및 베테랑 주의사항
`{% extends 'base.html' %}`	현재 자식 템플릿이 상속받을 부모 스켈레톤 템플릿을 지정함	**반드시 HTML 파일의 가장 첫 번째 줄(1번 줄)에 위치해야 함!**
`{% block content %}`	부모 템플릿에서 자식 템플릿이 내용을 덮어쓸 수 있도록 뚫어놓은 영역	`{% block content %} ... {% endblock content %}` (자식에서 `block` 밖에 쓴 코드는 무시됨)
`{% for %} ... {% empty %}`	리스트/QuerySet을 반복 출력하며, 비어 있을 때는 `{% empty %}` 아래 문구를 출력함	`{% for a in articles %} {{ forloop.counter }} {% empty %} 작성된 글이 없습니다. {% endfor %}`
`forloop` 내장 객체	`{% for %}` 반복문 내부에서 자동으로 제공되는 반복 순서 카운터 객체	• `{{ forloop.counter }}`: 1부터 시작하는 번호 • `{{ forloop.counter0 }}`: 0부터 시작 / `{{ forloop.first }}`: 첫 번째 반복이면 `True`
`{% if %}` / `{% elif %}` / `{% else %}`	조건이 참일 때만 해당 HTML 영역을 렌더링함 (`and`, `or`, `not`, `==`, `in` 사용 가능)	`{% if request.user == article.user %} [수정] {% endif %}` (**주의: 괄호 `()` 사용 불가!**)

DTL 태그 / 필터 문법	핵심 역할 및 출력 결과	실전 코드 예시 및 베테랑 주의사항
`{% csrf_token %}`	CSRF 공격 방어를 위해 `<input type="hidden" name="csrfmiddlewaretoken" value="...">`를 생성	모든 `<form method="POST">` 내부에 필수 삽입! (누락 시 `403 Forbidden` 에러)
필터 `|length` / `|default`	• `|length`: 문자열이나 리스트의 길이(개수) 반환 • `|default:'값'`: 변수가 비어있거나 `False`일 때 기본값 출력	전체 게시글: `{{ articles|length }}`개 `{{ comment|default:'댓글 없음' }}`
필터 `|truncatechars:N` 필터 `|date:"Y-m-d"`	• `|truncatechars:N`: N글자까지만 자르고 `...` 붙임 (단어 기준은 `truncatewords`) • `|date:"형식"`: 날짜/시간 객체를 지정한 포맷으로 출력	`{{ article.content|truncatechars:20 }}` `{{ article.created_at|date:"Y년 m월 d일 H:i" }}`

테마 04 : Model 필드 타입 · 필드 옵션 · 마이그레이션 필수 용어 (교재 3권)

필드 / 옵션 / 명령어	핵심 정의 및 데이터베이스 역할	실전 문법 예시 및 베테랑 암기 포인트
Schema (스키마)	데이터베이스에서 자료의 구조, 표현 방법, 관계 등을 정의한 전반적인 명세(설계도)	`models.py`의 클래스 하나 = DB 테이블 스키마 하나
Primary Key (`pk` / `id`)	테이블의 각 행(레코드)을 유일하게 식별하는 고유 번호표 컬럼 (Django가 `id` 필드를 자동 생성함)	`article.id`와 `article.pk`는 완전히 동일하게 고유 번호를 가리킴
`CharField(max_length=N)`	길이 제한이 있는 문자열 필드 (DB의 `VARCHAR(N)`으로 변환됨)	**`max_length` 인자가 없으면 `makemigrations` 시 즉시 에러 발생!**
`TextField()`	길이 제한이 없는 대용량 텍스트 필드 (DB의 `TEXT`로 변환됨)	게시글 본문, 자기소개 등에 사용 (`max_length`는 필수가 아님)
`DateTimeField` 옵션: `auto_now_add` vs `auto_now`	• `auto_now_add=True`: 최초 생성(INSERT) 시점의 날짜·시각을 영구 고정 • `auto_now=True`: 데이터가 저장(`save()`)될 때마다 현재 시각으로 갱신	암기 공식: **Add**(처음 추가할 때만 도장!) vs **Now**(지금 저장할 때마다 갱신!)
필드 옵션: `null=True` vs `blank=True`	• `null=True`: DB 컬럼에 `NULL` 값 저장 허용 (DB 수준) • `blank=True`: 폼 입력창을 비워둔 채 제출해도 유효성 검사 통과 (Form 수준)	**[베테랑 철칙]** `CharField`·`TextField`에는 `null=True`를 쓰지 말고 `blank=True`만 사용!
`makemigrations` vs `migrate`	• `makemigrations`: `models.py` 변경사항을 읽어 `migrations/` 폴더에 파이썬 설계도 생성 • `migrate`: 설계도를 SQL로 번역해 실제 `db.sqlite3`에 테이블 반영	암기 공식: `makemigrations`(설계도 인쇄 — Git 공유 O) → `migrate`(실제 땅에 시공)

테마 05 : UI/UX 3대 법칙 · 웹 폰트 · 반응형 웹 필수 용어 (교재 4권)

필수 용어 / 속성명	핵심 정의 및 원리	적용 방법 및 베테랑 1줄 암기 공식
UI vs UX	• **UI (User Interface)**: 화면 레이아웃, 버튼, 색상 등 사용자가 마주하는 시각적 도구 • **UX (User Experience)**: 서비스를 이용하며 느끼는 총체적 경험과 만족감	"예쁜 메뉴판은 UI, 음식 나올 때까지의 편리함과 만족감은 UX!"
힉의 법칙 (Hick's Law)	선택지의 개수와 복잡도가 증가할수록 사용자가 결정을 내리는 시간이 길어져 이탈한다는 법칙	해결책: 핵심 기능만 먼저 보여주고 단계별로 펼치는 **점진적 공개(Progressive Disclosure)**
피츠의 법칙 (Fitts's Law)	목표물까지의 거리가 가깝고 목표물의 크기가 클수록 빠르고 정확하게 조작할 수 있다는 법칙	해결책: 스마트폰 하단 **엄지 영역(Thumb Zone)**에 48×48px 이상의 큼직한 버튼 배치

필수 용어 / 속성명	핵심 정의 및 원리	적용 방법 및 베테랑 1줄 암기 공식
스켈레톤 UI (Skeleton UI)	데이터 로딩 중 단순 회전 스피너 대신 실제 화면의 회색 윤곽선 뼈대를 먼저 보여주는 기법	"사용자의 체감 대기 시간을 줄여주는 심리적 UX 마법"
FOIT vs FOUT (`font-display: swap`)	• **FOIT**: 웹 폰트 다운로드 전 글자가 아예 안 보이는 현상 • **FOUT**: 기본 글꼴을 먼저 보여주고 다운로드 직후 웹 폰트로 바꾸는 방식	`font-display: swap;`을 설정해 글자가 증발하는 FOIT를 막고 FOUT 방식으로 부드럽게 전환!
`viewport` **메타 태그**	스마트폰 브라우저가 자신을 980px 데스크톱 모니터로 착각해 화면을 축소하는 것을 막고, 기기의 실제 가로 폭(`device-width`)에 맞추도록 지시하는 태그	`<meta name="viewport" content="width=device-width, initial-scale=1.0">` (반응형 웹의 필수 전제 조건!)
Media Query (`@media`) `min-width` vs `max-width`	화면 너비 등 기기 특성에 따라 서로 다른 CSS 스타일을 적용하는 반응형 핵심 문법	• `min-width: 768px` = 768px **이상**일 때 (모바일 퍼스트, 작은 폭→큰 폭 오름차순 작성!) • `max-width: 768px` = 768px **이하**일 때 (내림차순 작성)

테마 06 : ORM · QuerySet API · Field Lookups 필수 용어 (교재 5권)

필수 용어 / 메서드명	핵심 정의 및 반환값 특징	코드 예시 및 베테랑 암기 공식
ORM (Object-Relational Mapping)	파이썬 객체(Object)와 관계형 데이터베이스(SQL) 사이를 양방향으로 번역해 주는 기술	SQL을 직접 짜지 않아도 파이썬 코드로 모든 DB 조작 가능
Manager (`.objects`)	Model 클래스가 데이터베이스 쿼리 작업(QuerySet API)을 호출할 수 있도록 연결해 주는 기본 인터페이스 창구	`Article.objects.all()`에서 한가운데의 `.objects`
QuerySet (쿼리셋)	DB에서 전달받은 객체들의 목록(리스트 형태 바구니). 인덱싱·슬라이싱·반복문 순회가 가능함	`all()`, `filter()`, `exclude()`, `order_by()`는 무조건 `QuerySet`을 반환함!
지연 평가 (Lazy Evaluation)	QuerySet을 만드는 코드 줄에서는 DB에 SQL을 보내지 않고, 실제로 데이터를 출력·순회할 때 단 한 번 DB를 조회하는 최적화 성질	여러 필터 조건을 체이닝해도 마지막에 최적화된 SQL 1개로 합쳐서 실행됨
`filter()` **vs** `get()`	• `filter()`: 조건에 맞는 객체들을 0개든 1개든 N개든 무조건 `QuerySet` **바구니**로 반환 • `get()`: 바구니 없이 **단 1개의 모델 인스턴스(알맹이)**만 직접 반환	`get()`은 결과가 0개면 `DoesNotExist`, 2개 이상이면 `MultipleObjectsReturned` 에러가 터지므로 **오직 `pk` 조회에만 사용!**
`order_by('필드')`	QuerySet의 정렬 순서를 지정하는 메서드. 필드명 앞에 하이픈(`-`)을 붙이면 내림차순(최신순) 정렬	`Article.objects.order_by('-pk')` (최신 글부터 정렬)
Field Lookups (`__`)	`filter()`, `exclude()`, `get()` 안에서 SQL의 `WHERE` 상세 조건을 지정할 때 쓰는 이중 밑줄(`__`) 문법	`title__contains='장고'`, `pk__gt=5`, `title__startswith='공지'`

테마 07 : Django Form · ModelForm · 유효성 검사 필수 용어 (교재 7권)

필수 용어 / 메서드명	핵심 정의 및 역할	코드 예시 및 베테랑 암기 공식
`forms.Form` **vs** `forms.ModelForm`	• `Form`: 입력 데이터를 DB 모델에 그대로 저장하지 않을 때 사용 (검색, 로그인, 비밀번호 변경) • `ModelForm`: 입력 데이터를 DB 테이블에 곧바로 생성·수정할 때 사용 (게시글, 회원가입, 회원수정)	DB 저장 직행열차 = `ModelForm` / 단순·특수 검문소 = `Form`

필수 용어 / 메서드명	핵심 정의 및 역할	코드 예시 및 베테랑 암기 공식
`class Meta:` (ModelForm 내부)	`ModelForm`이 어느 Model을 바탕으로 어떤 입력 필드들을 만들지 등록하는 설계 메타 정보 클래스	`model = Article` / `fields = '__all__'` 또는 `exclude = ('author',)`
Widget (위젯)	HTML `<input>`이나 `<textarea>` 태그의 표현 방식과 CSS 클래스, placeholder 등의 속성을 제어하는 도구	`forms.TextInput(attrs={'class': 'form-control'})` (**주의: 유효성 검사와는 무관하며 순수 화면 출력만 담당!**)
`form.is_valid()`	제출된 폼 데이터가 모든 필드 조건(빈칸 여부, 최대 길이, 형식 등)을 만족하는지 검사하여 `True` / `False`를 반환하는 핵심 검문 메서드	통과 실패(`False`) 시 `form.errors` 안에 실패 원인 에러 메시지가 자동으로 채워짐!
`instance` 키워드 인자	`ModelForm`에서 새 행 생성(INSERT)이 아니라 **기존 DB 객체를 수정(UPDATE)**할 때 대상 객체를 넘겨주는 인자	GET 수정창: `ArticleForm(instance=article)` POST 수정저장: `ArticleForm(request.POST, instance=article)`
`form.save(commit=False)`	폼 데이터를 DB에 즉시 저장하지 않고, 한 템포 멈춰서 파이썬 메모리상의 모델 인스턴스 객체만 먼저 반환하는 옵션	`article = form.save(commit=False)` 후 `article.user = request.user`를 채우고 `article.save()` 실행!

테마 08 : 쿠키·세션 · Django Authentication(인증) 필수 용어 (교재 8·9·10권)

필수 용어 / 함수·클래스명	핵심 정의 및 작동 원리	코드 예시 및 베테랑 암기 공식
Connectionless & Stateless	• **비연결성(Connectionless)**: 서버가 응답 직후 연결을 끊는 성질 • **무상태성(Stateless)**: 연결을 끊어 이전 요청 상태를 기억하지 못하는 성질	"HTTP의 단기 기억상실증 → 이를 극복하려고 쿠키와 세션을 사용함!"
Cookie (쿠키) vs Session (세션)	• **쿠키**: 클라이언트 브라우저에 저장되는 작은 Key-Value 데이터 조각 • **세션**: 서버 DB(`django_session`)에 중요 인증 정보를 저장하고, 브라우저 쿠키에는 금고 열쇠(`sessionid`)만 발급하는 방식	암기 공식: 쿠키 = 손님 수첩 속 금고 열쇠(`sessionid`) / 세션 = 서버 DB 속 보안 금고
`AbstractUser` & `AUTH_USER_MODEL`	기본 `auth.User`와 동일한 기능을 가지면서 향후 컬럼 확장이 자유로운 추상 부모 클래스와 이를 프로젝트 기본 유저로 등록하는 설정값	반드시 **첫 `migrate` 전에** `class User(AbstractUser): pass` 및 `AUTH_USER_MODEL = 'accounts.User'` 설정!
`get_user_model()`	현재 프로젝트의 `AUTH_USER_MODEL`에 등록된 활성 User 모델 클래스를 동적으로 찾아 반환하는 함수	`models.py`에서는 문자열 `settings.AUTH_USER_MODEL`을 쓰고, 나머지 모든 파일(`forms.py` 등)에서는 `get_user_model()`을 사용!
`AuthenticationForm` vs `UserCreationForm`	• `AuthenticationForm`: 로그인용 내장 `Form` (제1인자: `request`) • `UserCreationForm`: 회원가입용 내장 `ModelForm` (제1인자: `request.POST`)	회원가입·회원수정은 `ModelForm`이므로 커스텀 상속 필수! 로그인·비번변경은 일반 `Form`이므로 그대로 사용!
`UserChangeForm` & `ReadOnlyPasswordHashField`	회원정보 수정용 내장 `ModelForm`. 기본적으로 관리자용(`fields='__all__'`)이라 모든 권한 필드가 노출되며, 하단에 `/accounts/password/` 링크를 안내함	반드시 `CustomUserChangeForm`에서 `fields = ('first_name', 'last_name', 'email')`로 일반 필드만 노출되도록 제한 필수!

필수 용어 / 함수·클래스명	핵심 정의 및 작동 원리	코드 예시 및 베테랑 암기 공식
`PasswordChangeForm` & `SetPasswordForm`	비밀번호 변경용 내장 `Form`. 부모 클래스인 `SetPasswordForm`의 생성자 규칙을 따르므로 **제1인자로 대상 `user` 객체**를 받음	GET: `PasswordChangeForm(request.user)` POST: `PasswordChangeForm(request.user, request.POST)`
`update_session_auth_hash(request, user)`	비밀번호 변경 시 DB 암호 해시값이 바뀌어 현재 세션이 무효화(자동 로그아웃)되는 것을 막기 위해 새 해시로 세션을 갱신하는 함수	`user = form.save()` 바로 아랫줄에 `update_session_auth_hash(request, user)` 필수 호출! (타 기기 해커 세션은 로그아웃됨)
`is_authenticated` & `@login_required`	• `request.user.is_authenticated`: 로그인 여부를 `True/False`로 알려주는 속성(괄호 없음!) • `@login_required`: 비로그인 사용자를 로그인 페이지(`?next=원래주소`)로 이동시키는 데코레이터	회원 탈퇴 시에는 반드시 ① `request.user.delete()` 먼저 → ② `auth_logout(request)` 나중 순서 엄수!

테마 09 : 비밀번호 암호화(해시·솔트·키 스트레칭) · 비밀번호 초기화 필수 용어 (교재 10권)

필수 용어 / 보안 기술명	핵심 정의 및 보안 작동 원리 (What & Why)	실전 구조 및 베테랑 1줄 암기 공식
크레덴셜 스터핑 (Credential Stuffing)	한 사이트에서 유출된 평문 아이디·비밀번호 목록을 다른 여러 사이트에 무작위로 대입해 연쇄 해킹하는 공격 기법	대부분의 사용자가 여러 사이트에서 같은 비밀번호를 재사용하기 때문에 평문 저장 시 치명적 2차 피해 발생!
인코딩(Encoding) vs 암호화(Encryption) vs 해시(Hash)	• **인코딩(Base64)**: 비밀키 없이 누구나 즉시 원래 값으로 디코딩 가능 (보안 효과 없음) • **암호화**: 비밀키로 복호화 가능한 양방향 변환 • **해시**: 원본 복원이 불가능한 **단방향 고정 길이 변환**	암기 공식: 인코딩 = 모양만 바꾼 평문 / 암호화 = 열쇠 있는 자물쇠(양방향) / **해시 = 과일 갈아 만든 주스(단방향, 비밀번호 저장용!)**
해시 함수 3대 특성 & 눈사태 효과 (Avalanche Effect)	① **고정된 길이 출력**(비밀번호 길이 은폐 및 저장 일관성) / ② **단방향성**(역추적 불가) / ③ **눈사태 효과**: 입력값이 점 하나만 달라져도 결과 해시값이 완전히 바뀌는 특성	`1234`와 `1235`는 단 한 글자 차이지만 해시 결과는 100% 전혀 다른 문자열이 출력됨!
`SHA-256`	어떤 크기의 데이터를 넣어도 항상 **256비트(16진수 64글자)**의 고정 해시값을 출력하는 표준 해시 알고리즘	역계산은 불가능하지만 **연산 속도가 너무 빠르다**는 점이 비밀번호 저장에서는 해커에게 유리한 약점이 됨!
레인보우 테이블 공격 (Rainbow Table)	해커가 자주 쓰이는 비밀번호 수억 개(`123456`, `qwer1234` 등)의 해시값을 미리 계산해 만들어 둔 거대한 '해시 정답지 사전'으로 대조하는 공격	역방향 복호화를 하지 않고도 미리 계산된 표에서 같은 해시값을 검색해 원래 비밀번호를 알아냄!
솔트 (Salt, 소금 치기)	**레인보우 테이블 공격을 방어**하기 위해 각 사용자마다 고유하게 생성된 임의의 문자열(솔트)을 비밀번호에 덧붙여 해싱하는 기법	같은 `qwer1234`라도 유저마다 솔트가 달라 해시값이 전부 달라짐! (해커가 솔트를 알아도 유저 1명당 새 표를 만들어야 해서 공격 무력화)

필수 용어 / 보안 기술명	핵심 정의 및 보안 작동 원리 (What & Why)	실전 구조 및 베테랑 1줄 암기 공식
무차별 대입 공격 (Brute-force Attack)	솔트 때문에 미리 만든 정답지를 못 쓰게 되자, 최신 GPU의 빠른 연산 속도(초당 수백억 번)로 가능한 모든 비밀번호를 하나씩 대입하는 공격	시간과의 싸움이며, 해시 함수의 빠른 연산 속도가 공격자의 가장 큰 무기가 됨!
키 스트레칭 (Key Stretching)	**무차별 대입 공격을 방어**하기 위해 솔트를 적용한 해시 함수를 수만~수십만 번 연속 반복하여 1회 검증 연산 시간을 의도적으로 늦추는(약 0.2초) 기법	정상 유저에게 0.2초는 체감되지 않지만, 해커가 10억 개 대입 시 **17분(1회) → 3.2년 이상(100만 회)**으로 지연됨!
`PBKDF2` (`pbkdf2_sha256`) & 4단 저장 포맷	Django가 기본으로 사용하는 키 스트레칭+솔트 해시 알고리즘 (더 강력한 **`Argon2`**, **`bcrypt`**도 지원). DB에는 달러(`$`)로 구분된 4단 문자열로 저장됨	포맷: `<algorithm>$<iterations>$<salt>$<hash>` **★ 주의: 사용자 아이디(`username`)는 절대 포함되지 않음!**
`django.contrib.auth.urls` & `EMAIL_BACKEND`	비밀번호 초기화 (**`password_reset`**) 등 내장 인증 View/URL 모음과, 개발 중 이메일을 터미널 콘솔에 출력해 주는 설정	• 루트 **`urls.py`**: 우리 **`accounts.urls`**를 내장 **`auth.urls`**보다 **윗줄**에 배치! • **`settings.py`**: **`EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'`**

테마 10 : PJT-04 Model · Form · Auth 실전 심화 필수 용어 (교재 11권)

필수 용어 / 메서드·옵션명	핵심 정의 및 작동 원리 (What & Why)	실전 코드 문법 및 베테랑 1줄 암기 공식
`choices` 속성 & `get_FOO_display()`	모델 필드에 **`(DB 저장값, 화면 표시 이름)`** 튜플 리스트로 선택지를 부여하는 속성과, 저장값 대신 화면용 이름을 꺼내주는 자동 생성 메서드	파이썬: **`todo.get_priority_display()`** 템플릿: **`{{ todo.get_priority_display }}`** (자판기 코드 vs 버튼 이름!)
`TextChoices` & `IntegerChoices`	선택지를 리스트 대신 클래스(Enum)로 묶어 **`Status.DOING`**처럼 이름으로 안전하게 참조해 오타를 막아주는 클래스	**`class Status(models.TextChoices): TODO = 'TODO', '할 일'`** → **`choices=Status.choices`**
`PositiveIntegerField`	0 이상의 정수(양수 및 0)만 저장하도록 강제하며, DB 테이블에도 **`CHECK (>= 0)`** 제약을 생성하는 숫자 필드	**`choices`**(검증 단계 1차 방어) + **`PositiveIntegerField`**(DB 단계 2차 방어) = 완벽한 이중 방어!
`verbose_name` & `help_text`	• **`verbose_name`**: Admin/Form에서 필드의 라벨(이름표)로 표시될 텍스트 • **`help_text`**: 입력칸 옆이나 아래에 표시되는 안내 문구	**`status = models.CharField(..., verbose_name='상태', help_text='작업 상태를 고르세요.')`**
`form.cleaned_data`	**`form.is_valid()`**(보안 검색대)를 통과한 뒤, 단순 문자열이던 입력값들이 **알맞은 파이썬 자료형(`int`, `list` 등)으로 변환되어 담긴 깨끗한 딕셔너리**	반드시 **`if form.is_valid():`** 블록 안쪽에서만 꺼내 쓸 수 있음! (**`form.cleaned_data['tags']`**)
`Meta.widgets` vs 필드 재선언	• **`Meta.widgets`**: **`ModelForm`**에서 기존 모델 필드의 모양만 바꿈(모델 설정 유지) • **필드 재선언**: 모델에 없는 칸(**`keyword`**, **`password1`** 등)을 정의(기존 설정 초기화됨)	암기 공식: **`Meta.widgets`** = 방 벽지(인테리어)만 바꾸기 / 필드 재선언 = 가구 새로 사기(**`label`** 재지정 필수!)

필수 용어 / 메서드·옵션명	핵심 정의 및 작동 원리 (What & Why)	실전 코드 문법 및 베테랑 1줄 암기 공식
`MultipleChoiceField` & `CheckboxSelectMultiple`	여러 선택지를 동시에 골라 파이썬 리스트(`['WORK', 'STUDY']`)로 받는 폼 필드와, 이를 체크박스로 그려주는 위젯	저장 시: `','.join(form.cleaned_data['tags'])` 수정 창 복원 시: `self.initial['tags'] = self.instance.get_tag_list()`
Django Message Framework (`messages`)	저장·수정·비밀번호 변경 후 `redirect()`된 다음 화면에서 **딱 한 번만 보여주고 사라지는 일회용 알림** (5등급: `debug`, `info`, `success`, `warning`, `error`)	View: `messages.success(request, '메시지')` Template: `{% for message in messages %}` ("읽으면 떼는 포스트잇!")
`url_has_allowed_host_and_scheme` & `|urlencode`	• `?next=` 이동 전 우리 사이트 주소인지 검사해 **오픈 리다이렉트(피싱 사이트 납치) 공격을 막는 함수** • `|urlencode`: `?next=` 값 안의 `?`, `&`, 한글을 `%XX`로 변환하는 필터	택시 출발 전 우리 동네 목적지인지 확인하는 검문소! 템플릿에선 `?next={{ request.GET.next|urlencode }}` 사용
`password = None` & `user_profile` 분리	• `CustomUserChangeForm` 상단에 `password = None`을 적어 비밀번호 해시 안내 칸 제거 • 프로필 뷰에서 `context` 키를 `'user_profile'`로 넘겨 기본 `{{ user }}`(로그인 유저)와 충돌 방지	프로필 뷰에서 `{'user': user}`로 넘기면 상단 내비게이션 바의 로그인 유저 정보가 바뀌어버리는 버그 예방!
`@require_POST` 데코레이터	로그아웃·회원탈퇴·삭제 뷰를 링크나 주소창(`GET`)으로 호출하지 못하게 막고 `POST` 요청만 허용하는 데코레이터	외부 사이트의 `<img src="/accounts/logout/">` 이미지 태그 공격을 원천 차단 (위반 시 `405` 반환)
커스텀 User 2대 실전 에러 (`E304` & `Manager isn't available`)	• `fields.E304`: `User` 클래스만 만들고 `AUTH_USER_MODEL` 설정을 빼먹어 `user_set` 역참조가 겹칠 때 발생 • `Manager isn't available`: `UserCreationForm`을 상속·교체하지 않고 기본 폼 그대로 가입 시도 시 발생	1) `settings.py`에 `AUTH_USER_MODEL = 'accounts.User'` 등록 필수! 2) 회원가입·수정은 반드시 `Meta.model = get_user_model()`로 교체!

부록 : 시험 직전 10초 만에 정답을 골라내는 베테랑의 핵심 비교 공식 5선

단골 출제 비교 항목	A 선택지 (정확한 쓰임새)	B 선택지 (정확한 쓰임새 및 함정 주의)
회원 모델 참조 위치	`settings.AUTH_USER_MODEL`: 오직 `models.py`에서 외래키·M:N 관계를 정의할 때만 문자열로 사용	`get_user_model()`: `forms.py`, `views.py` 등 `models.py`를 제외한 모든 파이썬 파일에서 사용
인증 폼 상속 필요 여부	**상속 재정의 필수 (`ModelForm`)**: `UserCreationForm`(가입), `UserChangeForm`(수정 — `fields` 제한 필수)	**그대로 import 사용 (일반 `Form`)**: `AuthenticationForm`(로그인), `PasswordChangeForm`(비번 변경)
해시 공격 vs 방어 기법	**레인보우 테이블 공격**(미리 계산한 해시 정답지 사전) → **솔트(Salt, 사용자별 무작위 문자열 추가)**로 방어!	**무차별 대입 공격(Brute-force)**(고속 GPU 전수 대입) → **키 스트레칭(Key Stretching, `PBKDF2` 반복 연산)**으로 방어!
회원 탈퇴 vs 비번 변경	**회원 탈퇴 순서**: 반드시 ① `request.user.delete()` 먼저 실행 후 → ② `auth_logout(request)` 실행	**비밀번호 변경 후 세션 유지**: `user = form.save()` 직후 반드시 `update_session_auth_hash(request, user)` 실행
비밀번호 해시 저장 포맷	**포함되는 4요소**: `<algorithm>$<iterations>$<salt>$<hash>` (알고리즘, 반복 횟수, 솔트, 해시값)	**절대 포함되지 않는 요소**: 사용자 아이디(`username`)나 원본 비밀번호 길이는 해시 문자열에 들어가지 않음!